



We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech

Contents

01

Joins

02

**Database Objects -
Views and Indexes**

Joins



Joins

Introduction

- **Join** operation is used to create a **connection between two or more relations** based on common attributes.
- The tables are related to each other using the **primary key** and **foreign key relationship**, but it is **not mandatory**.
- It **combines rows** from various tables based on the matching column.
- Used to establish a **relationship** between the **relations**.
- **Types of Joins:** Inner Join, Outer Join, Cross Join
- The **inner join** is divided into **four types**: EQUI join, NON-EQUI join, SELF-join, Natural Join
- The **outer join** is divided into three types: Left outer join, Right outer join, Full outer join

Joins

Introduction

Syntax :

SELECT

ColumnList

FROM LeftTableName

JOIN_TYPE RightTableName

ON JoinCondition;

Joins

Introduction

- Let us understand the examples using the following tables

Department Table:

	DepartmentID	Name	Building
▶	D01	Computer Science	CS BLOCK
	D02	Electronics & Communication	ECE BLOCK
	D03	Information Technology	IT BLOCK
	D04	EEE	EEE BLOCK

Course Table:

CourseID	Title	Credits	DepartmentID
CS101	Data Structures	4	D01
CS102	DBMS	3	D01
CS103	Operating Systems	3	D01
EC101	Digital Circuits	3	D02
EC102	Analog Electronics	3	D02
IT101	C Programming	5	D03
IT102	Problem Solving	2	D03

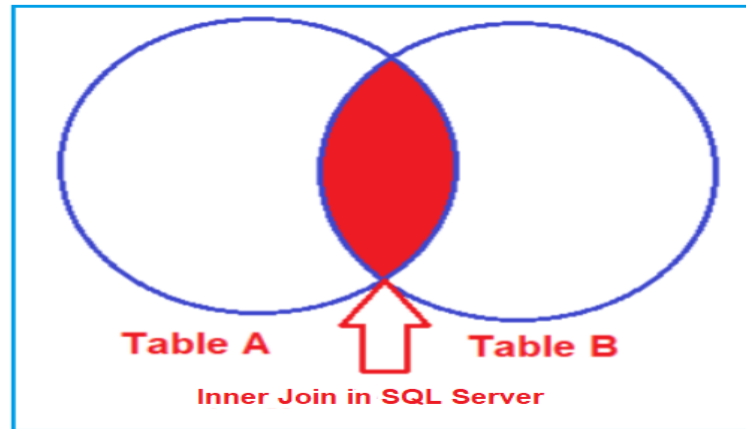
Professor Table:

ProfessorID	Name	Email	DepartmentID
P01	Dr. Kumar	kumar@csuniv.edu	D01
P02	Dr. Meena	meena@csuniv.edu	D01
P03	Dr. Raj	raj@eceuniv.edu	D02
P04	Dr. Leela	leela@eceuniv.edu	D02
P05	Dr. Ravi	ravi@csuniv.edu	D01

Joins

MySQL Inner Join

- **Inner Join** selects tuples that have **matching values** in **both tables** based on the given condition.
- Returns the combination of all tuples that satisfies the given condition from both relations.



Syntax:

```
SELECT column_name(s) FROM table1 INNER JOIN / JOIN table2  
ON table1.column_name = table2.column_name;
```

Joins

MySQL Inner Join

- Instead of using the INNER JOIN keyword, the **JOIN keyword** is also used because the **default joins** in MySQL is **Inner Join**.
- The Inner Join in MySQL is used to **return** only the **matching records** from **both the tables** involved in the join and the **non-matching records** are simply **ignored** or **eliminated**.

Example #1:

Display the courses offered by the department by matching the DepartmentID column

```
SELECT CourseID, Title, Credits, Department.DepartmentID, Name
```

```
from Department INNER JOIN Course
```

```
on Department.DepartmentID = Course.DepartmentID;
```

OUTPUT

	CourseID	Title	Credits	DepartmentID	Name
▶	CS101	Data Structures	4	D01	Computer Science
	CS102	DBMS	3	D01	Computer Science
	CS103	Operating Systems	3	D01	Computer Science
	EC101	Digital Circuits	3	D02	Electronics & Communication
	EC102	Analog Electronics	3	D02	Electronics & Communication
	IT101	C Programming	5	D03	Information Technology
	IT102	Problem Solving	2	D03	Information Technology

Joins

MySQL Inner Join : Equi Join

- SQL **EQUI JOIN** performs a **JOIN against equality** or **matching column(s) values** of the associated tables.
- An **equal sign (=)** is used as a **comparison operator** in the **where clause** to refer to equality.
- EQUI JOIN is performed by using **JOIN keyword followed** by the **ON keyword** and then specifying the names of the columns along with their associated tables to **check equality**.

Syntax 1:

```
SELECT column_name(s) FROM table1, table2  
WHERE table1.column_name = table2.column_name;
```

Syntax 2:

```
SELECT * FROM table1 JOIN table2 [ON (join_condition)];
```

Joins

MySQL Inner Join : Equi Join : Example

- The following example uses the **= sign** in the **where clause** to get the **rows from Course table** that has the corresponding rows with the **same values** in the DepartmentID column of **Department** table.

Example #2:

```
SELECT CourseID, Title, Credits, Department.DepartmentID, Name
FROM Course, Department
WHERE Department.DepartmentID = Course.DepartmentID;
```

OUTPUT

	CourseID	Title	Credits	DepartmentID	Name
▶	CS101	Data Structures	4	D01	Computer Science
	CS102	DBMS	3	D01	Computer Science
	CS103	Operating Systems	3	D01	Computer Science
	EC101	Digital Circuits	3	D02	Electronics & Communication
	EC102	Analog Electronics	3	D02	Electronics & Communication
	IT101	C Programming	5	D03	Information Technology
	IT102	Problem Solving	2	D03	Information Technology

Joins

MySQL Inner Join : Equi Join : Example

- Instead of using the = sign in the where clause, the **JOIN keyword** is also used as shown in the below Query.

Example #3:

```
SELECT CourseID, Title, Credits, Department.DepartmentID, Name  
FROM Course JOIN Department  
ON Department.DepartmentID = Course.DepartmentID;
```

OUTPUT

	CourseID	Title	Credits	DepartmentID	Name
▶	CS101	Data Structures	4	D01	Computer Science
	CS102	DBMS	3	D01	Computer Science
	CS103	Operating Systems	3	D01	Computer Science
	EC101	Digital Circuits	3	D02	Electronics & Communication
	EC102	Analog Electronics	3	D02	Electronics & Communication
	IT101	C Programming	5	D03	Information Technology
	IT102	Problem Solving	2	D03	Information Technology

Joins

MySQL Inner Join : Non Equi Join

- The SQL **NON EQUI JOIN** uses a **comparison operator** instead of the equal sign like **>**, **<**, **>=**, **<=** along with conditions.
- The **value of the column** in **each row** from the **source table** is **compared** with the corresponding value of the **target table**.
- If the **data matches** the source and target table, the comparison returns **true**, and therefore that **data** is **retrieved from the table**.

Syntax:

```
SELECT * FROM table_name1, table_name2
```

```
WHERE table_name1.column [ > | < | >= | <= ] table_name2.column;
```

Joins

MySQL Inner Join : Non Equi Join : Example

- The following example uses the **comparison operator** in the **where clause** to get the **rows** from **Course** table and **Department** table.
- The condition **Credits > 2** is a **non-equi filter condition** - filters based on a range (non-equi logic).

Example #4:

```
SELECT CourseID, Title, Credits, Department.DepartmentID, Name
FROM Course, Department
WHERE Department.DepartmentID=Course.DepartmentID
AND Credits>2;
```

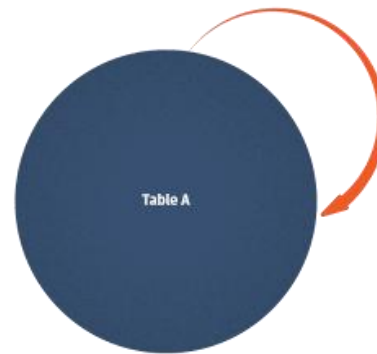
OUTPUT

	CourseID	Title	Credits	DepartmentID	Name
▶	CS101	Data Structures	4	D01	Computer Science
	CS102	DBMS	3	D01	Computer Science
	CS103	Operating Systems	3	D01	Computer Science
	EC101	Digital Circuits	3	D02	Electronics & Communication
	EC102	Analog Electronics	3	D02	Electronics & Communication
	IT101	C Programming	5	D03	Information Technology

Joins

MySQL Inner Join : Self Join

- A **self join** is a join in which a **table is joined with itself** (which is also called Unary relationships), especially when the table has a FOREIGN KEY that references its own PRIMARY KEY.
- To **join a table itself** means that each row of the table is combined with itself and with every other row of the table.
- The self join can be **viewed** as a **join of two copies of the same table**.
- The table is **not actually copied**, but SQL performs the command as though it were.



Joins

MySQL Inner Join : Self Join

- The **syntax** of the command for **joining a table to itself** is **almost the same** as that for **joining two different tables**.
- To **distinguish** the **column names** from one another, **aliases** for the **actual table name** are used, since both the tables have the same name.
- Useful for hierarchical data, peer comparison, or grouping within the same category.
- Table name aliases are **defined** in the **FROM clause** of the SELECT statement.

Syntax:

```
SELECT a.column_name, b.column_name FROM table_name1 a, table_name1 b  
WHERE a.column_name=b.column_name;
```

Joins

MySQL Inner Join : Self Join : Example

- **The following example to Find Employees Working in the Same City.** You want to identify pairs of employees working in the same city.
- To achieve this, you can perform a self-join on the Employee table by matching the City column.

Example #5:

```
SELECT P1.ProfessorID AS Prof1_ID, P1.Name AS Prof1_Name,  
P2.ProfessorID AS Prof2_ID, P2.Name AS Prof2_Name,  
P1.DepartmentID FROM Professor P1 JOIN Professor P2  
ON P1.DepartmentID = P2.DepartmentID  
AND P1.ProfessorID < P2.ProfessorID;
```

OUTPUT

	Prof1_ID	Prof1_Name	Prof2_ID	Prof2_Name	DepartmentID
►	P01	Dr. Kumar	P02	Dr. Meena	D01
	P01	Dr. Kumar	P05	Dr. Ravi	D01
	P02	Dr. Meena	P05	Dr. Ravi	D01
	P03	Dr. Raj	P04	Dr. Leela	D02

Joins

MySQL Inner Join : Self Join : Example

Explanation:

- P1 and P2 are aliases of the same Professor table.
- P1.DepartmentID = P2.DepartmentID: Ensures both professors are in the same department.
- P1.ProfessorID < P2.ProfessorID: avoids duplicate pairs and self-pairing (like Dr. Kumar with Dr. Kumar).

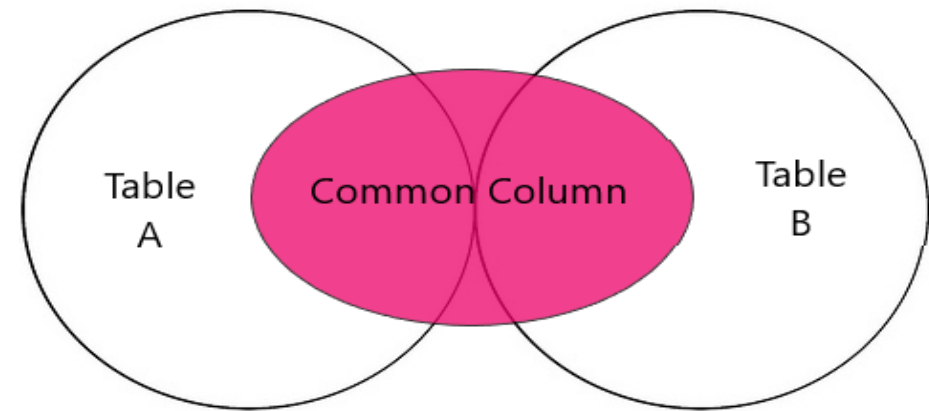
Joins

MySQL Inner Join : Natural Join

- The SQL **NATURAL JOIN** is a type of EQUI JOIN and is structured in such a way that, **columns** with the **same name** of **associated tables** will **appear once only**.
- The **associated tables** have **one or more pairs** of **identically named columns**.
- The **columns** must be the **same data type**.
- This does **not use** the **ON clause** in a natural join.

Syntax:

```
SELECT column_name(s)
FROM
table_name1 NATURAL JOIN table_name2;
```



Joins

MySQL Inner Join : Natural Join : Example

Example #6:

```
SELECT * FROM Course NATURAL JOIN Department;
```

OUTPUT

	DepartmentID	CourseID	Title	Credits	Name	Building
▶	D01	CS101	Data Structures	4	Computer Science	CS BLOCK
	D01	CS102	DBMS	3	Computer Science	CS BLOCK
	D01	CS103	Operating Systems	3	Computer Science	CS BLOCK
	D02	EC101	Digital Circuits	3	Electronics & Communication	ECE BLOCK
	D02	EC102	Analog Electronics	3	Electronics & Communication	ECE BLOCK
	D03	IT101	C Programming	5	Information Technology	IT BLOCK
	D03	IT102	Problem Solving	2	Information Technology	IT BLOCK

- Here, Course and Department table have DepartmentID as **identically named columns** and **DepartmentID appeared only once** in the output.

Joins

MySQL Outer Join

- The **OUTER JOIN** in MySQL returns matched data rows as well as unmatched data rows from both the tables involved in the join.
- Outer join in MySQL is again classified into **three types** as follows:
 - Left outer join
 - Right outer join
 - Full outer join

Joins

MySQL Outer Join : Left Outer Join

- The **LEFT OUTER JOIN** in MySQL is used to **retrieve** all the **matching rows** from both the tables involved in the join as well as non-matching rows from the left side table.
- The **un-matching data** will take a **null value**.
- The table mentioned to the left of the **Left outer join keyword** is the **left table**, and the table mentioned to the **right** of the **Left outer join keyword** is the **right table**.
- The following diagram is the pictorial representation of MySQL Left Outer Join.



Joins

MySQL Outer Join : Left Outer Join

- The **LEFT OUTER JOIN** or **LEFT JOIN** keyword is used to perform Left Outer Join.

Syntax 1:

```
SELECT column_name(s) FROM table1 LEFT OUTER JOIN table2  
ON table1.column_name = table2.column_name;
```

Syntax 2:

```
SELECT column_name(s) FROM table1 LEFT JOIN table2  
ON table1.column_name = table2.column_name ;
```

Joins

MySQL Outer Join : Left Outer Join : Example

- Returns all records from the left table (Professor), and the matched records from the right table (Department). If there's no match, NULLs are shown for right table columns.
- **For each professor, their department's name and building will be shown if a match is found.**

Example #7:

```
SELECT P.ProfessorID, P.Name, D.Name AS DeptName, D.Building  
FROM Professor P LEFT OUTER JOIN Department D ON P.DepartmentID = D.DepartmentID;
```

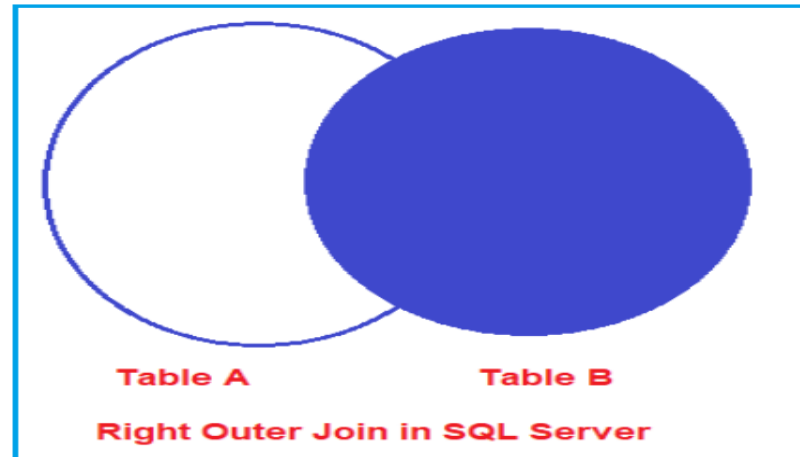
OUTPUT

	ProfessorID	Name	DeptName	Building
▶	P01	Dr. Kumar	Computer Science	CS BLOCK
	P02	Dr. Meena	Computer Science	CS BLOCK
	P03	Dr. Raj	Electronics & Communication	ECE BLOCK
	P04	Dr. Leela	Electronics & Communication	ECE BLOCK
	P05	Dr. Ravi	Computer Science	CS BLOCK

Joins

MySQL Outer Join : Right Outer Join

- MySQL **right join** returns all the **values from relation at right** and the **matching values from the left relation**.
- The **RIGHT OUTER JOIN** in MySQL is used to **retrieve all the matching rows from both the tables** involved in the join as well as **non-matching rows from the right-side table**.
- Returns **NULL** when there is **no match exists**.



Joins

MySQL Outer Join : Right Outer Join

- In MySQL, the **RIGHT OUTER JOIN** or **RIGHT JOIN** keyword is used to perform Right Outer Join.

Syntax 1:

```
SELECT column_name(s) FROM table1 RIGHT OUTER JOIN table2  
ON table1.column_name = table2.column_name;
```

Syntax 2:

```
SELECT column_name(s) FROM table1 RIGHT JOIN table2  
ON table1.column_name = table2.column_name;
```

Joins

MySQL Outer Join : Right Outer Join : Example

- Returns all records from the right table (Department), and the matched records from the left table (Professor). If there's no match, NULLs are shown for left table columns.
- If a department has no professor assigned, then ProfessorID and Name will be NULL.

Example #8:

```
SELECT P.ProfessorID, P.Name, D.Name AS DeptName, D.Building  
FROM Professor P RIGHT OUTER JOIN Department D ON P.DepartmentID = D.DepartmentID;
```

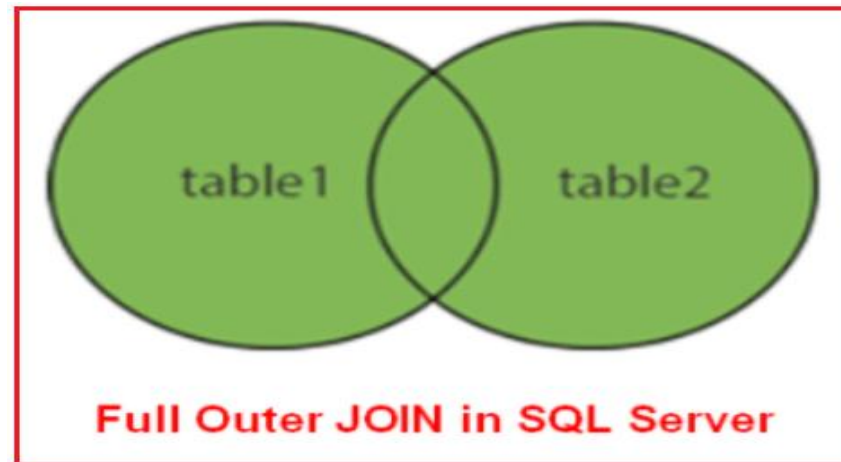
OUTPUT

ProfessorID	Name	DeptName	Building
P05	Dr. Ravi	Computer Science	CS BLOCK
P02	Dr. Meena	Computer Science	CS BLOCK
P01	Dr. Kumar	Computer Science	CS BLOCK
P04	Dr. Leela	Electronics & Communication	ECE BLOCK
P03	Dr. Raj	Electronics & Communication	ECE BLOCK
NULL	NULL	Information Technology	IT BLOCK
NULL	NULL	EEE	EEE BLOCK

Joins

MySQL Outer Join : Full Outer Join

- Full join Combination of both **left** and **right** outer join returns all the records from both the relations satisfying the condition.
- The Full Outer Join in MySQL is used to **retrieve** all the **matching and non-matching records** from both the tables involved in the JOIN.
- Rows without match from either side will have **NULLs for the missing part**.



Joins

MySQL Outer Join : Full Outer Join

- **FULL OUTER JOIN** or **FULL JOIN** keyword is used to perform Full Outer Join.

Syntax:

```
SELECT column_name(s) FROM table1 FULL OUTER JOIN table2  
ON table1.column_name = table2.column_name WHERE condition;
```

Syntax:

```
SELECT column_name(s) FROM table1 FULL JOIN table2  
ON table1.column_name = table2.column_name WHERE condition;
```

- MySQL does not support **FULL OUTER JOIN** directly, but it can be emulated using **UNION**

Joins

MySQL Outer Join : Full Outer Join : Example

Example #9: Combines the results of both LEFT and RIGHT joins

```
SELECT P.ProfessorID, P.Name, D.Name AS DeptName, D.Building
FROM Professor P LEFT JOIN Department D ON P.DepartmentID = D.DepartmentID
```

UNION

```
SELECT P.ProfessorID, P.Name, D.Name AS DeptName, D.Building
FROM Professor P RIGHT JOIN
Department D ON P.DepartmentID = D.DepartmentID;
```

OUTPUT

ProfessorID	Name	DeptName	Building
P01	Dr. Kumar	Computer Science	CS BLOCK
P02	Dr. Meena	Computer Science	CS BLOCK
P03	Dr. Raj	Electronics & Communication	ECE BLOCK
P04	Dr. Leela	Electronics & Communication	ECE BLOCK
P05	Dr. Ravi	Computer Science	CS BLOCK
NULL	NULL	Information Technology	IT BLOCK
NULL	NULL	EEE	EEE BLOCK

Joins

MySQL Cross Join

- The **CROSS JOIN** is created by using the CROSS JOIN keyword.
- In Cross Join, **each record** of a **table** is **joined** with **each record** of the **other table**.
- In MySQL, the Cross Join **should not have** either an **ON** or **where clause**.
- A Cross Join in MySQL **produces** the **Cartesian product** of the **tables** involved in the join.
- The **Cartesian product** means the **number of records present** in the **first table** is **multiplied by** the **number of records present** in the **second table**.

Syntax:

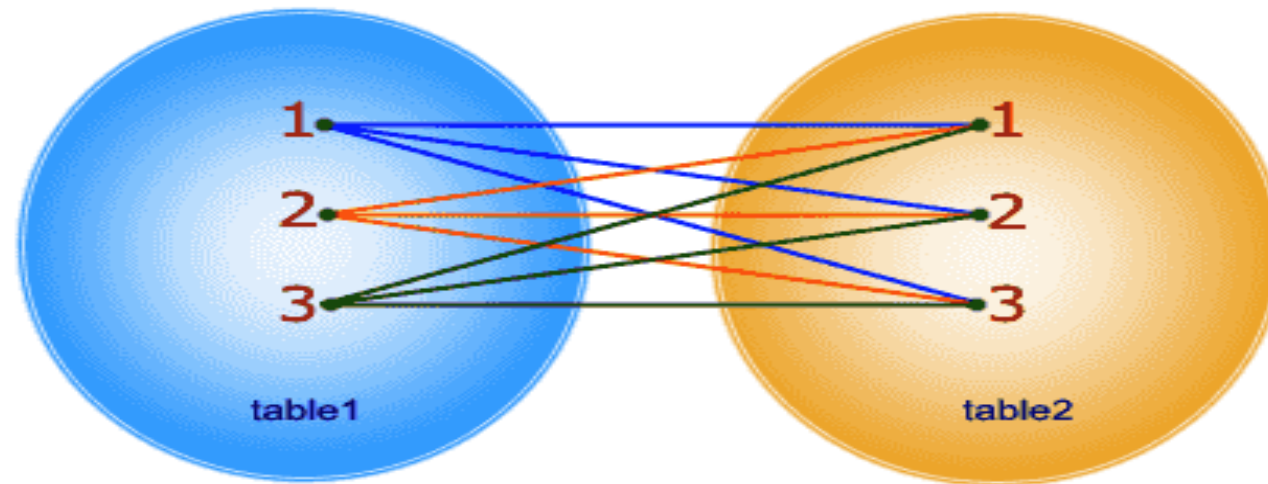
```
SELECT column_name(s) FROM table1 CROSS JOIN table2;
```

Joins

MySQL Cross Join

- The following diagram is the pictorial representation of MySQL Cross Join.

`SELECT * FROM table1 CROSS JOIN table2;`



In CROSS JOIN, each row from 1st table joins with all the rows of another table.
 If 1st table contain x rows and y rows in 2nd one the result set will be $x * y$ rows.

Joins

MySQL Cross Join : Example

Example #10:

```
SELECT P.ProfessorID, P.Name, D.Name AS
DeptName, D.Building
FROM Professor P CROSS JOIN Department D;
```

- Here, there are **5 professors** and **4 departments**, the result contains $5 \times 4 = \mathbf{20 \text{ rows}}$.
- Every professor is matched with every department, regardless of any logic.

OUTPUT

ProfessorID	Name	DeptName	Building
P01	Dr. Kumar	EEE	EEE BLOCK
P01	Dr. Kumar	Information Technology	IT BLOCK
P01	Dr. Kumar	Electronics & Communication	ECE BLOCK
P01	Dr. Kumar	Computer Science	CS BLOCK
P02	Dr. Meena	EEE	EEE BLOCK
P02	Dr. Meena	Information Technology	IT BLOCK
P02	Dr. Meena	Electronics & Communication	ECE BLOCK
P02	Dr. Meena	Computer Science	CS BLOCK
P03	Dr. Raj	EEE	EEE BLOCK
P03	Dr. Raj	Information Technology	IT BLOCK
P03	Dr. Raj	Electronics & Communication	ECE BLOCK
P03	Dr. Raj	Computer Science	CS BLOCK
P04	Dr. Leela	EEE	EEE BLOCK
P04	Dr. Leela	Information Technology	IT BLOCK
P04	Dr. Leela	Electronics & Communication	ECE BLOCK
P04	Dr. Leela	Computer Science	CS BLOCK
P05	Dr. Ravi	EEE	EEE BLOCK
P05	Dr. Ravi	Information Technology	IT BLOCK
P05	Dr. Ravi	Electronics & Communication	ECE BLOCK
P05	Dr. Ravi	Computer Science	CS BLOCK

Joins

Join Multiple Tables in MySQL

- We use **multiple table** joins to **combine data from more than two tables**.
- The join operator is used **multiple times to join** multiple tables in SQL, as for each new table, one join is added.
- In SQL, joining multiple tables means you can join n number of tables, but to join **n number of tables**, the joining required is **n-1** that is for 3 tables, 2 joins are required.

Syntax: For SQL Join 3 tables

```
SELECT column-names FROM table-name1
```

```
JOIN table-name2 ON column-name1 = column-name2
```

```
JOIN table-name3 ON column-name3 = column-name4
```

```
WHERE condition
```

Joins

Join Multiple Tables in MySQL

Syntax: For SQL Join n tables

SELECT column-names

FROM table-name1

JOIN table-name2 ON column-name1 = column-name2

JOIN table-name3 ON column-name3 = column-name4

JOIN table-name4 ON column-name5 = column-name6

...

WHERE condition

Joins

Join Multiple Tables in MySQL : using INNER JOIN

- In SQL, an inner join obtains common rows from all of the tables mentioned in a query.
- In our case, if we see the **Student** and **Branch table** have Br_id common and the **Address and Student table** have city_id common.
- So, to retrieve data, first, we need to join two tables and then the third table. Let's take an example to make it more clear.

Example: Obtain Students' names along with their branch names, HOD, City, and Pincode

```
SELECT Student.Name, Branch.Br_id, Branch.HOD,  
Address.city, Address.pincode from Branch  
INNER JOIN Student  
on Student.Br_id = Branch.Br_id  
INNER JOIN Address  
on Student.city_id = Address.city_id;
```

Joins

Join Multiple Tables in MySQL : using INNER JOIN

- The output of the above example is

Name	Br_id	HOD	City	Pincode
Ankit	101	SH Rao	Mumbai	400121
Pranav	105	VL Shelke	Pune	450011
Raj	102	AP Sharma	Pune	450011
Jhon	102	AP Sharma	Lucknow	553001
Aman	101	SH Rao	Delhi	443221

Joins

Join Multiple Tables in MySQL : using LEFT JOIN

- When we use **left join on the multiple tables**, it will include **all the rows from the left table**, that is the table mentioned on the left side of the join and **the next left joins will be applied to the result of the first left join**.

Example: Obtain all the names of the students even if they are not present in any City or Branch.

```
Select Student.Name,  
Address.city,  
Branch.Br_name from Student  
left join Branch on Student.Br_id = Branch.Br_id  
left join address on Student.City_id = Address.City_id;
```

Joins

Join Multiple Tables in MySQL : using LEFT JOIN

- The output of the above example will be

Name	City	Br_name
Ankit	Mumbai	CSE
Pranav	Pune	IT
Raj	Pune	MECH
Shyam	Delhi	NULL
Duke	Pune	NULL
Jhon	Lucknow	MECH
Aman	Delhi	CSE
Pavan	NULL	NULL
Virat	NULL	NULL

Joins

Join Multiple Tables in MySQL : using LEFT JOIN

- In the **left join on the multiple tables example**, the **first left join** gets **all the rows** with the names of students from the **Student table** along with **all the Br_name**.
- If the **Name is not present** in any **branch** the **Br_name field**, we get for that **Name is NULL**.
- On this table, **another left join** is applied on the **Address table** which keeps all the **Name and Br_name** as it is and returns **all rows from the city column**.
- If the **Name and Br_name** are not present in any City the City field we get for those **Name and Br_name is Null**.

Joins

Join Multiple Tables in MySQL : using both INNER JOIN and LEFTJOIN

- The combination of **inner join** and **left join** can be used in SQL queries where the **order of join decides the result of the query**.

Example: Obtain all the Student Names even if they are not present in any Branch whereas excluding the Name which is not present in any city.

```
Select Student.Name,  
Address.city,  
Branch.Br_name from Student  
left join Branch  
on Student.Br_id = Branch.Br_id  
inner join address  
on Student.City_id = Address.City_id;
```

Joins

Join Multiple Tables in MySQL : using both INNER JOIN and LEFTJOIN

- In this query, first all the names from the *Student* table are returned and the Br_name is returned even if the name does not have any Br_name by using left join. On this result, an inner join is applied with the *Address* table which eliminates the name with no city.
- **The output of the above query will be as follows,**

Name	City	Br_name
Ankit	Mumbai	CSE
Pranav	Pune	IT
Raj	Pune	MECH
Shyam	Delhi	NULL
Duke	Pune	NULL
Jhon	Lucknow	MECH
Aman	Delhi	CSE

Joins

When to use Subqueries vs. Joins

Scenario	Prefer Subqueries	Prefer Joins
Filtering Data	Use subqueries for dynamic filtering.	Use joins for combining related datasets.
Aggregations	Ideal for nested calculations (e.g., MAX, AVG).	Joins are less common but usable for group data.
Complex Relationships	Better for handling correlated subqueries.	Best for straightforward relationships.
Performance Considerations	Subqueries can be slower, especially nested ones.	Joins are faster with proper indexing.

Joins

Order of Clauses

- **Execution Order (Logical Flow) vs Writing Order (Syntax Order) are different —** but let's start with the syntax order you must write in:

-- Syntax Order (How You Write It)

SELECT

[DISTINCT] column_list

FROM

table_name

[JOIN table_name ON condition]

[WHERE condition]

[GROUP BY column_list]

[HAVING condition]

[ORDER BY column_list [ASC|DESC]]

[LIMIT number [OFFSET number]];

Joins

Order of Clauses

-- Example with Your Database

--Task: Get the top 3 Computer Science students, grouped by semester, who have an average grade above 80, sorted by semester.

```
SELECT DISTINCT S.Name, E.Semester, AVG(E.Grade) AS AvgGrade
FROM Student S
JOIN Enrollment E ON S.StudentID = E.StudentID
WHERE S.Major = 'Computer Science'
GROUP BY S.Name, E.Semester
HAVING AVG(E.Grade) > 80
ORDER BY E.Semester ASC
LIMIT 3;
```

Joins

Order of Clauses

Logical Execution Order (How MySQL Processes It)

- Even though you write SELECT first, MySQL processes the query in this order:
- FROM → Identify source tables
- JOIN ... ON → Combine data from multiple tables
- WHERE → Filter rows before grouping
- GROUP BY → Group rows for aggregation
- HAVING → Filter aggregated results
- SELECT → Choose columns or computed values
- DISTINCT → Remove duplicate rows from the result
- ORDER BY → Sort the final results
- LIMIT → Restrict number of rows returned

Joins

Quiz



1. Which of the following returns only the rows where there is a match in both tables?

a) LEFT JOIN

b) RIGHT JOIN

c) INNER JOIN

d) FULL OUTER JOIN

Answer : Option C)

Joins

Quiz



2. In a LEFT JOIN, which rows are always included in the result?

a) Only matching rows from both tables

b) All rows from the right table, and matched rows from the left

c) All rows from the left table, and matched rows from the right

d) Only unmatched rows from both tables

Answer : Option C)

Quiz



3. MySQL doesn't directly support FULL OUTER JOIN.
Which of the following can simulate it?

a) LEFT JOIN only

b) RIGHT JOIN only

c) CROSS JOIN

d) LEFT JOIN UNION RIGHT JOIN

Answer : Option d)

Quiz



4. What does a CROSS JOIN in MySQL produce?

a) Cartesian product of two tables

b) Only common rows between two tables

c) Only distinct rows from two tables

d) Union of two tables

Answer : Option a)

Quiz



5. A NATURAL JOIN automatically joins tables based on:

a) All primary keys

b) All columns with the same names in both tables

c) The first column in both tables

d) User-specified join condition

Answer : Option b)

Database Objects



Database Objects

What are database objects?

- In MySQL, a **database object** is any **defined structure** that is used to **store, manage, or reference data** within a **database**.
- These objects are part of the **database schema** and are **permanently stored in the database**.
- **Common Database Object in MySQL**
 - Tables
 - Views
 - Indexes
 - Stored Procedures
 - Triggers
 - Functions

Database Objects

Common Database Objects in MySQL : Tables

1. Tables:

- The most **fundamental database object** and used to **store data in rows and columns**.

-- Example : Creating the Employee Table

```
CREATE TABLE employees ( id INT PRIMARY KEY, name VARCHAR(50), salary DECIMAL(10, 2) );
```

Database Objects

Common Database Objects in MySQL: Views

2. Views:

- View is a **subset of one or more base tables** and also called as **virtual tables**.
- Stored as a **select statement** in the data dictionary .
- View will **not have data of its own** and will **not occupy space**.

Types of views:

- Simple Views
- Complex Views

Common Database Objects in MySQL: Views

Simple View:

- Simple View contains only a **single table** and cannot use **group functions and any groups of data**.
- View should not have **DISTINCT keyword**.

Syntax:

```
CREATE OR REPLACE VIEW view_name AS SELECT column_list FROM table_name.  
  
SELECT * FROM view_name;
```

-- Example: Creating View “vu_stud ”

```
CREATE OR REPLACE view vu_stud AS SELECT ename,email,address from student_details;  
  
SELECT * FROM vu_stud;
```

Database Objects

Common Database Objects in MySQL: Views

Read Only View:

- Restrict any DML operation.
- Use WITH **READ ONLY OPERATION**.

Syntax:

```
CREATE OR REPLACE VIEW view_name AS SELECT column_list FROM table_name  
  
WITH READ ONLY;
```

-- Example: Creating View “vu_stud ” with READ ONLY

```
CREATE OR REPLACE vu_stud_read AS SELECT ename,email,address from student_details  
  
WITH READ ONLY;
```

Database Objects

Common Database Objects in MySQL: Views

Complex View:

- **Complex View** contains **more than one table and** can have group functions and groups of data.
- **DML is not always possible through views.**

Syntax:

```
CREATE OR REPLACE VIEW view_name AS
```

```
SELECT t1,column,t2.column2 FROM table1 t1,table2 t2 WHERE t1.column1=t2.column1;
```

Common Database Objects in MySQL: Indexes

- Indexes are **special lookup tables** that the database search engine can use to speed up data retrieval.
- An index in a database is very similar to an **index in the back of a book**.
- **For Example:** if you want to **reference all pages** in a book that discusses a certain topic, you first refer to the **index**, which lists all the topics **alphabetically** and is then referred to **one or more** specific page numbers.
- An index helps to speed up **SELECT** queries and **WHERE** clauses, but it slows down data input, with the **UPDATE** and the **INSERT** statements.
- Indexes can be **created or dropped with no effect on the data**.

Common Database Objects in MySQL: Indexes

- Indexes are **special lookup tables** that the database search engine can use to speed up data retrieval.
- An index in a database is very similar to an **index in the back of a book**.
- **For Example:** if you want to **reference all pages** in a book that discusses a certain topic, you first refer to the **index**, which lists all the topics **alphabetically** and is then referred to **one or more** specific page numbers.
- An index helps to speed up **SELECT** queries and **WHERE** clauses, but it slows down data input, with the **UPDATE** and the **INSERT** statements.
- Indexes can be **created or dropped with no effect on the data**.

Database Objects

Common Database Objects in MySQL: Indexes

Consider the example 'Employee table'

- **Employeeid** is the primary key, so by default a **Primary index** on the **Employeeid** column is created.
- This means employee data is sorted by **Employeeid** column and **physically stored** in a series of **data pages** in a **tree like structure** that looks like the following.

Employees Table			
Employeeid	Name	Email	Department
1	Mark	mark@pragimtech.com	IT
2	John	john@pragimtech.com	HR
3	Sara	sara@pragimtech.com	HR
4	Mary	mary@pragimtech.com	IT
5	Dave	dave@pragimtech.com	IT
...			
...			
1200	Steve	steve@pragimtech.com	HR

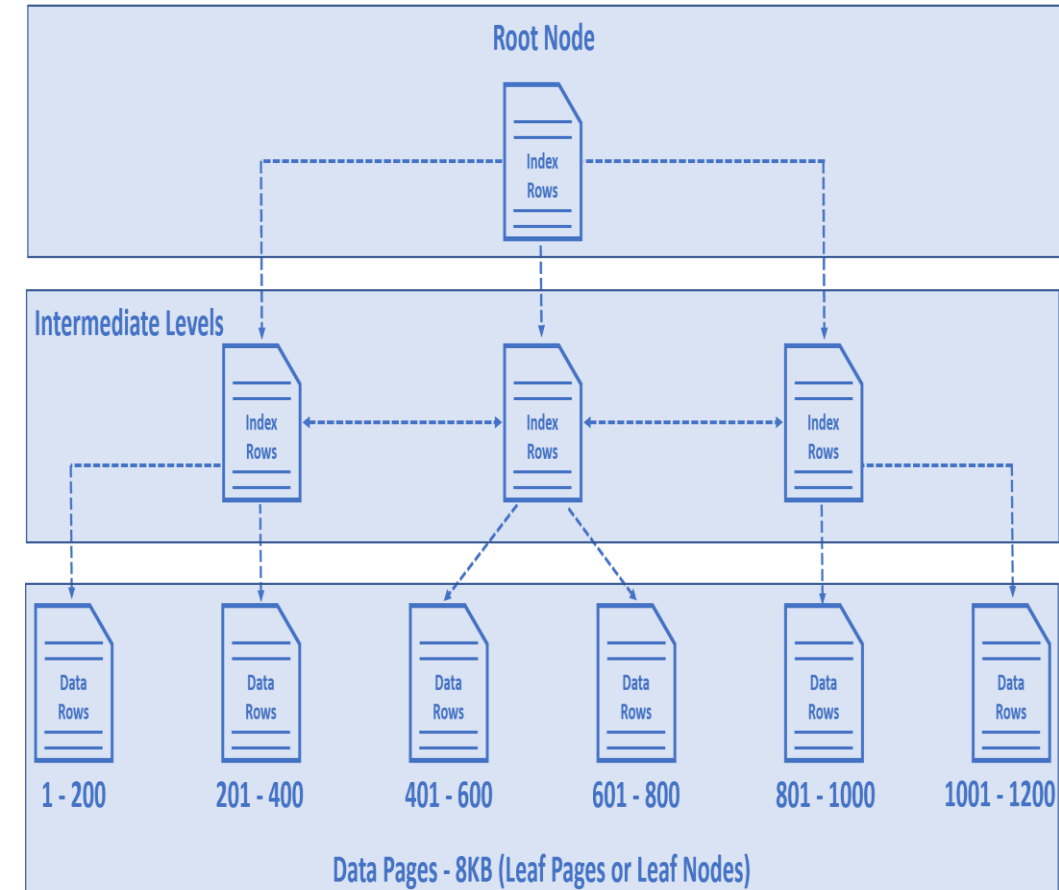
Primary Key

1200 rows in table

Database Objects

Common Database Objects in MySQL: Indexes

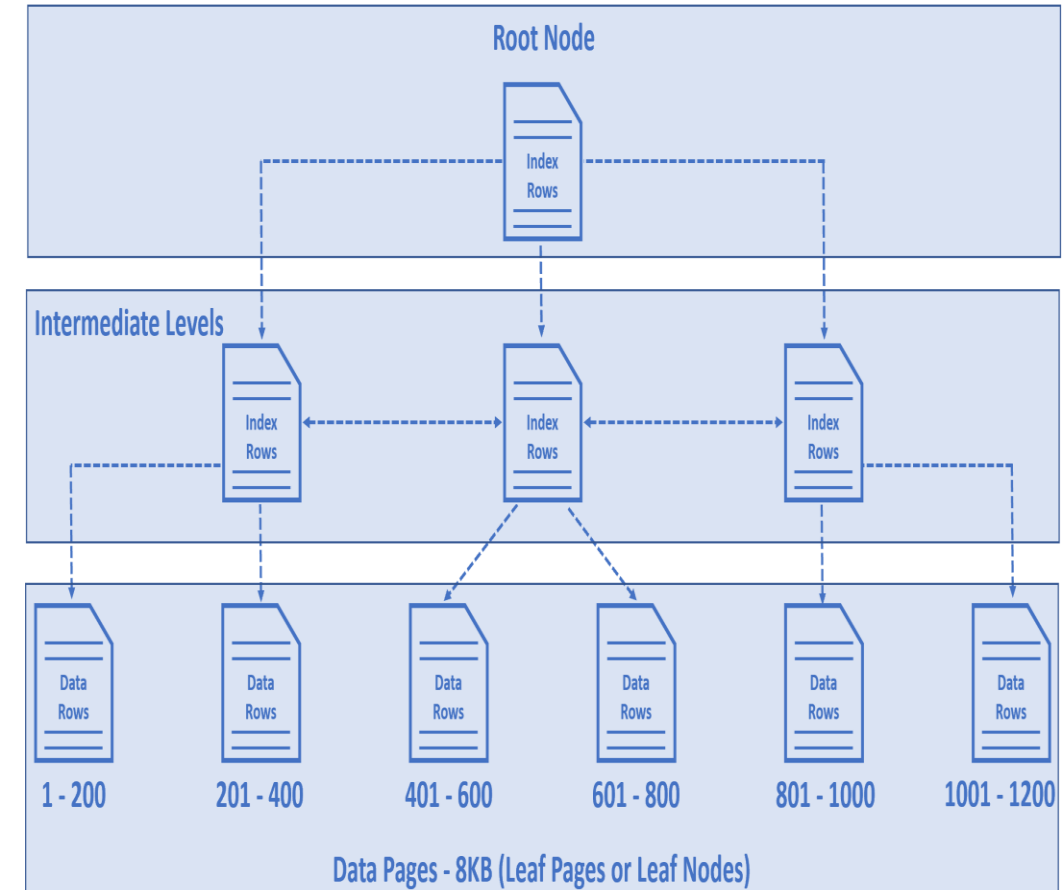
- The **nodes at the bottom** of the tree are called **data pages** or leaf nodes and **contain the actual data rows**, in our case **employee rows**.
- These **employee rows** are **sorted** by **EmployeeId** column, because **EmployeeId** is the **primary key** and by default a **primary index** on this **column** is created.
- For our example, let's say in **Employees table** we have **1200 rows** and let's assume in **each data page** we have **200 rows**.



Database Objects

Common Database Objects in MySQL: Indexes

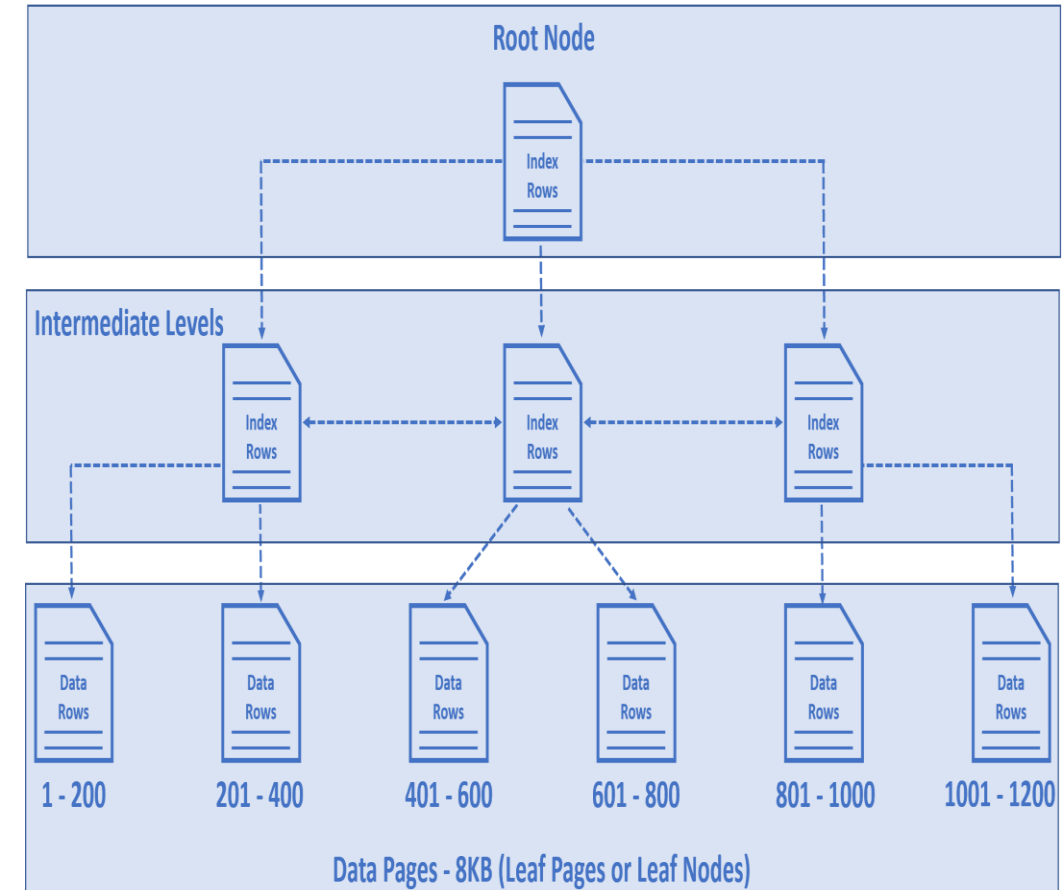
- So, in the **first data page** we have **1 to 200 rows**, in the **second 201 to 400**, in the **third 401 to 600**, so on and so forth.
- The node at the top of the tree is called **Root Node**.
- The **nodes between the root node** and the **leaf nodes** are called **intermediate levels**.
- The **root and the intermediate level nodes** contain **index rows**.



Database Objects

Common Database Objects in MySQL: Indexes

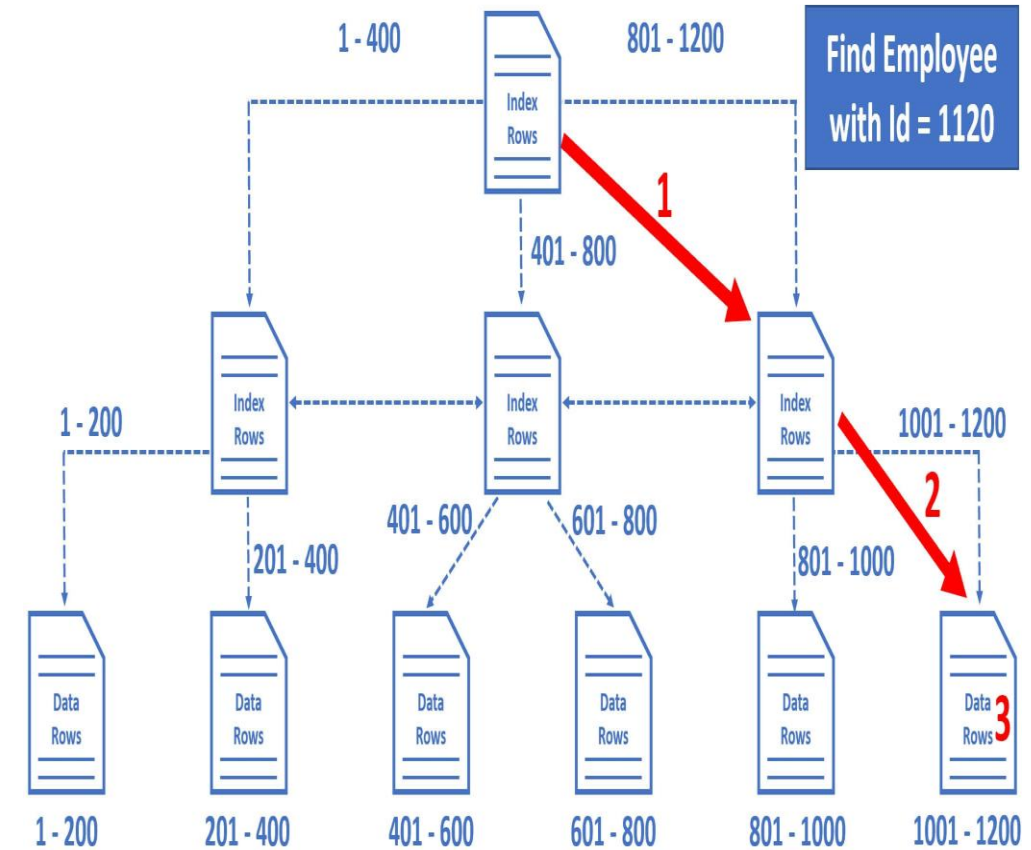
- Each **index row** contains a **key value** (in our case Employee Id) and a **pointer** to either an **intermediate level page in the B-tree**, or a **data row in the leaf node**.
- So this **tree like structure** has a **series of pointers** that helps the **query engine find data quickly**.



Database Objects

Common Database Objects in MySQL: Indexes

- To find the employee row with EmployeeId = 1120, the database engine starts at the root node.
- It selects the right index node, as it contains employee IDs from 801 to 1200.
- Next, it moves to the rightmost leaf node, which holds rows for IDs 1001 to 1200.
- Since the rows in the leaf node are sorted by Employee ID, it quickly locates the row with EmployeeId = 1120.



Common Database Objects in MySQL: Indexes

Index Creation Syntax

Simple Index:

```
CREATE INDEX index_name ON table_name(column_name);
```

Unique Index:

```
CREATE UNIQUE INDEX index_name ON table_name(column_name);
```

Composite Key Index:

```
CREATE INDEX index_name ON table_name(column1, column2);
```

Querying Index Information

```
SHOW INDEX FROM table_name;
```

Common Database Objects in MySQL: Indexes

Querying the Primary Indexing

Syntax:

```
SHOW INDEX FROM table_name;
```

Example:

```
CREATE TABLE employees (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(50),  
    department VARCHAR(50)  
);  
  
SHOW INDEX FROM employees;
```

Common Database Objects in MySQL: Indexes

Using information_schema.STATISTICS:

- The STATISTICS table in the information_schema database provides detailed index information.

Example:

```
SELECT TABLE_NAME, INDEX_NAME, COLUMN_NAME, NON_UNIQUE, SEQ_IN_INDEX,  
       INDEX_TYPE  
  
FROM information_schema.STATISTICS  
  
WHERE TABLE_SCHEMA = 'your_database_name' AND TABLE_NAME = 'employees';
```

Database Objects

Common Database Objects in MySQL: Triggers

- **Triggers** are **database objects** that **automatically execute predefined actions** when **specific events occur**.

Use Cases:

- Auditing changes in tables.
- Automatically updating related tables.

Types: BEFORE and AFTER for INSERT, UPDATE, DELETE.

Syntax:

```
CREATE TRIGGER trigger_name {BEFORE | AFTER} {INSERT | UPDATE | DELETE}
ON table_name FOR EACH ROW
BEGIN
    -- Trigger logic
END;
```

Common Database Objects in MySQL: Triggers

Example:

```
CREATE TRIGGER before_insert_student  
BEFORE INSERT ON students  
FOR EACH ROW  
SET NEW.created_at = NOW();
```

Database Objects

Common Database Objects in MySQL: Stored Procedure

- A **stored procedure** is a **group of SQL statements stored** in the database and **executed as a single unit**.

Advantages:

- Encapsulation and reuse of code.
- Improved performance and security.

Syntax:

```
CREATE PROCEDURE procedure_name(IN param1 TYPE, OUT param2 TYPE)

BEGIN

    -- Procedure body

END;
```

Common Database Objects in MySQL: Stored Procedure

DELIMITER //

```
CREATE PROCEDURE add_student(IN student_name VARCHAR(50), OUT message VARCHAR(100))
```

```
BEGIN
```

```
    INSERT INTO students (name) VALUES (student_name); -- First SQL statement
```

```
    SET message = 'Student added successfully';      -- Second SQL statement
```

```
END// -- This indicates the end of the procedure definition
```

```
DELIMITER ; -- Reverts back to the default semicolon delimiter
```

Note: The DELIMITER command in MySQL is used to temporarily change the statement delimiter from the default semicolon (;) to another string (like //, \$\$, or any other sequence).

Common Database Objects in MySQL: Functions

- **User-defined functions (UDFs)** are **reusable SQL code** that **returns a single value**.

Syntax:

```
CREATE FUNCTION function_name(parameters) RETURNS data_type  
  
BEGIN  
    -- Function logic  
  
    RETURN value;  
  
END;
```

Common Database Objects in MySQL: Functions

Example:

```
DELIMITER $$
```

```
CREATE FUNCTION calculate_discount(price DECIMAL(10,2))
```

```
RETURNS DECIMAL(10,2)
```

```
BEGIN
```

```
    RETURN price * 0.1;
```

```
END$$
```

```
DELIMITER ;
```

Quiz



1. What is an advantage of using views?

a) Improve query performance always

b) Provide security by restricting column access

c) Reduce storage space since views do not store data, only definitions

d) Both b and c

Answer : Option d)

Quiz



2. An index in MySQL is mainly used for:

a) Storing duplicate values

b) Improving query performance

c) Preventing data loss

d) Compressing data

Answer : Option b)

Quiz



3. A Trigger in a database is:

a) Manual query

b) Automatically executes on events (INSERT, UPDATE, DELETE)

c) Virtual table

d) Index

Answer : Option b)

Quiz



4. A Stored Procedure in a database is:

a) A precompiled set of SQL statements stored in the database

b) A way to store large files

c) A virtual table

d) A function that returns only one value

Answer : Option a)

Quiz



5. A Function in a database differs from a Stored Procedure because:

a) Functions can return a single value and be used in SQL expressions

b) Functions cannot accept parameters

c) Functions can perform DDL operations

d) Functions are never stored in the database

Answer : Option a)

THANK YOU